LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 06 - INHERITANCE



Nama : Gendis Pambayun

NM: 103132400031

Kelas: PS1SD-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Inheritance atau pewarisan adalah suatu proses di mana sebuah kelas diturunkan dari kelas lainnya sehingga mendapatkan ciri atau sifat (baik state maupun behaviour) dari kelas tersebut. Hubungan ini mencerminkan hubungan "is-a" (adalah bagian dari) antar objek. Tujuan utama dari adanya pewarisan yaitu menghindari duplikasi objek, baik berupa *field*, variabel, maupun *method* yang sebenarnya dapat diturunkan dari satu kelas umum ke kelas yang lebih spesifik.

Terminologi terkait:

a. Superclass (Kelas Dasar): Kelas yang menurunkan karakteristik atau sifat.
b. Subclass (Kelas Turunan): Kelas yang menerima penurunan sifat dari superclass dan bersifat lebih spesifik. Seluruh karakteristik superclass otomatis dimiliki oleh subclass-nya.

Method overloading dan overriding:

a. Method Overloading adalah kemampuan untuk membuat dua atau lebih method dengan nama yang sama di dalam satu kelas. Syarat utamanya adalah jumlah, urutan, atau tipe data argumen/parameter dari method-nya harus berbeda.
b. Method Overriding terjadi ketika sebuah subclass mendefinisikan kembali sebuah method dengan nama dan tanda (signature) yang sama persis dengan yang sudah dimiliki oleh superclass-nya.

Polymorphism atau Polimorfisme adalah kemampuan suatu objek untuk mengambil berbagai bentuk yang berbeda. Konsep ini memungkinkan penggunaan interface atau referensi kelas induk yang sama untuk memerintah objek melakukan tindakan yang secara prinsip serupa, meskipun secara proses implementasinya berbeda.

B. Guided

1. Guided 1

a. A

```
package com.gendispambayun.modul06.guided.guided1;


public class A {
    int x;
    int y;


    void tampilXY(){
    System.out.println("Nilai X = "+x+", y = +y");


    }
}
```

b. B

```
package com.gendispambayun.modul06.guided.guided1;


public class B extends A {
    int z;
    int hasil;


    void jumlahXYZ(){
        hasil = (x+y+z);


        System.out.println("Jumlahh (x+y+z) = " + hasil);
    }
}
```

c. Demo

```
package com.gendispambayun.modul06.guided.guided1;

public class Demo {

    public static void main(String[] args){

        A  superclass = new A();
        B subclass  = new B();

        superclass.x = 3;
        subclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;
        System.out.println("Subclass: ");
        subclass.jumlahXYZ();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Modul06 ---
Superclass:
Nilai X = 3, y = +y
Subclass:
Jumlahh (x+y+z) = 8
------------------------------------------------
BUILD SUCCESS
```

2. Guided 2

    a. Employee

```
//employee superclass
package com.gendispambayun.modul06.guided.guided2;

public class Employee {
```

```
    private  String nama;
    String departemen;

    public Employee(String nama){
        this.nama = nama;
    }

    public void tampilNama(){
        System.out.println("nama: "+ nama);
    }
}
```

b. Manager

```
package com.gendispambayun.modul06.guided.guided2;

public class Manager extends Employee{
    private  String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();

        System.out.println("alamat: " + alamat);
        System.out.println("departemen: " + super.departemen);
    }
}
```

c. Demo

```
package com.gendispambayun.modul06.guided.guided2;

public class Demo {
    public static void main(String[] args){
        Manager gendis = new Manager("Gendis", "Purbalingga");
        gendis.departemen = "Finance";
        gendis.tampilInformasi();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Modul06 ---
nama: Gendis
alamat: Purbalingga
departemen: Finance
------------------------------------------------
BUILD SUCCESS
```

3. Guided 3

    a. Lagu

```
package com.gendispambayun.modul06.guided.guided3;

class Lagu {

    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void Tampilkan() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}
```

    b. Main

```
package com.gendispambayun.modul06.guided.guided3;
public class Main {

    public static void main(String[] args) {
        Lagu d, e;
        d = new Lagu();
        e = new Lagu();
        d.IsiParam("Lagu 1");
        e.IsiParam("kepastian yang kutunggu", "GiGi");
        d.Tampilkan();
```

```
        e.Tampilkan();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Modul06 ---
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : kepastian yang kutunggu, pencipta : GiGi
------------------------------------------------
BUILD SUCCESS
```

4. Guided 4

a. A

```
package com.gendispambayun.modul06.guided.guided4;

class A {

    public void tampilkanKeLayar() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

b. B

```
package com.gendispambayun.modul06.guided.guided4;

class B extends A {

    public void tampilkanKeLayar() {
        super.tampilkanKeLayar(); // Method milik superclas dipanggil
        System.out.println("Method milik class B dipanggil...");
    }
}
```

c. Main

```
package com.gendispambayun.modul06.guided.guided4;

class Main {
```

```
    public static void main(String[] args) {
        B subOb = new B();
        subOb.tampilkanKeLayar();
    }
}
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ Modul06 ---
Method milik class A dipanggil...
Method milik class B dipanggil...
------------------------------------------------
BUILD SUCCESS
```

C. Unguided
    1. Soal Unguided

Soal Unguided — Modul 6 Inheritance

Sebuah tim *machine learning* ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis *deep learning* yang merupakan pengembangan dari model klasifikasi biasa.

Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

Ketentuan

1. Kelas induk (model umum)

- Menyimpan **nama model** (teks) dan **jumlah data latih** (bilangan bulat).
- Memiliki *constructor* yang menerima kedua nilai tersebut, serta cara bagi kelas lain untuk membaca jumlah data latih.
- Memiliki method untuk menampilkan informasi.
- Memiliki dua method untuk menghitung biaya komputasi dengan nama yang sama (*overloading*):
    - Tanpa parameter: biaya = jumlah data latih × 2.
    - Dengan satu parameter bilangan bulat / jumlah epoch: biaya = jumlah data latih × epoch.

2. Kelas turunan pertama (model regresi)

- Mewarisi kelas induk dan tambahkan atribut **jumlah fitur**.
- *Constructor* menerima nama, jumlah data latih, dan jumlah fitur; gunakan *constructor* kelas induk.
- **Override** method: tampilkan informasi dari kelas induk.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) + (jumlah fitur × 10).
- Tambahkan **satu method baru** dengan nama yang sama dengan method penghitung biaya, tetapi menerima **dua parameter bilangan bulat** (jumlah *epoch* dan ukuran *batch*): biaya = jumlah data latih × epoch ÷ ukuran batch. Sehingga objek kelas ini memiliki **tiga versi** method penghitung biaya.

3. Kelas turunan kedua (model klasifikasi)

- Mewarisi kelas induk dan menambahkan atribut **jumlah kelas**.

- *Constructor* menerima nama, jumlah data latih, dan jumlah kelas; gunakan *constructor* kelas induk.
- **Override** method: tampilkan informasi dari kelas induk.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) × jumlah kelas.
- Versi penghitung biaya dengan parameter *epoch* **tidak** di-override.

4. Kelas turunan ketiga (model klasifikasi *deep learning*)

- Mewarisi **kelas turunan kedua** (bukan kelas induk langsung) dan menambahkan atribut **jumlah layer**.
- *Constructor* menerima nama, jumlah data latih, jumlah kelas, dan jumlah layer; gunakan *constructor* kelas di atasnya.
- **Override** method: tampilkan informasi dari kelas di atasnya.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas di atasnya) + (jumlah layer × 200).

5. Class Main

- Buat tiga objek berikut:

Jenis model	**Nama**	**Data latih**	**Atribut tambahan**
Regresi	"Regresi Linier"	500	4 fitur
Klasifikasi	"Decision Tree"	800	3 kelas
Klasifikasi *deep learning*	"Neural Network"	1000	5 kelas, 3 layer

- Simpan ketiga objek tersebut dalam satu array bertipe kelas induk.
- Gunakan perulangan untuk setiap elemen array: tampilkan informasi model, tampilkan biaya komputasi, lalu jumlahkan seluruh biaya.
- Setelah perulangan, tampilkan total biaya.
- Terakhir, tampilkan juga:
    - biaya 10 *epoch* untuk model "Neural Network" (ambil dari array),
    - biaya 10 *epoch* untuk model "Regresi Linier",
    - biaya 10 *epoch* dengan ukuran *batch* 50 untuk model "Regresi Linier".

Contoh Output

```
None
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

Jawaban :

a. ModelKomputasi

```
package com.gendispambayun.modul06.unguided;


public class ModelKomputasi {
    private String namaModel;
    private int jumlahDataLatih;


    public ModelKomputasi(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }


    public int getJumlahDataLatih() {
```

```
        return jumlahDataLatih;
    }

    public String getNamaModel() {
        return namaModel;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + namaModel + " | Data latih: " +
jumlahDataLatih);
    }

    public double hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    public double hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

b. ModelRegresi

```
package com.gendispambayun.modul06.unguided;

public class ModelRegresi extends ModelKomputasi {

    private int jumlahFitur;
```

```
    public ModelRegresi(String namaModel, int jumlahDataLatih, int jumlahFitur)
{
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public double hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    public double hitungBiaya(int epoch, int ukuranBatch) {
        return (double) (getJumlahDataLatih() * epoch) / ukuranBatch;
    }
}
```

c. ModelKlasifikasi

```
package com.gendispambayun.modul06.unguided;

public class ModelKlasifikasi extends ModelKomputasi {
```

```
    private int jumlahKelas;

    public ModelKlasifikasi(String namaModel, int jumlahDataLatih, int
jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    @Override
    public double hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

d. ModelDeepLearning

```
package com.gendispambayun.modul06.unguided;

public class ModelDeepLearning extends ModelKlasifikasi {

    private int jumlahLayer;
```

```
    public ModelDeepLearning(String namaModel, int jumlahDataLatih, int
jumlahKelas, int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    @Override
    public double hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

e. Main

```
package com.gendispambayun.modul06.unguided;

public class Main {

    public static void main(String[] args) {
```

```
        ModelKomputasi[] daftarModel = new ModelKomputasi[3];

        daftarModel[0] = new ModelRegresi("Regresi Linier", 500, 4);
        daftarModel[1] = new ModelKlasifikasi("Decision Tree", 800, 3);
        daftarModel[2] = new ModelDeepLearning("Neural Network", 1000, 5, 3);

        double totalBiaya = 0;

        for (ModelKomputasi model : daftarModel) {
            model.tampilkanInfo();
            double biaya = model.hitungBiaya();
            System.out.println("Biaya komputasi: " + (int) biaya);
            totalBiaya += biaya;
            System.out.println("---");
        }

        System.out.println("Total biaya: " + (int) totalBiaya);

        double biaya10EpochNN = daftarModel[2].hitungBiaya(10);
        System.out.println("Biaya 10 epoch (Neural Network): " + (int)
biaya10EpochNN);

        double biaya10EpochRegresi = daftarModel[0].hitungBiaya(10);
        System.out.println("Biaya 10 epoch (Regresi Linier): " + (int)
biaya10EpochRegresi);

        ModelRegresi regresiObj = (ModelRegresi) daftarModel[0];
```

```
        double biayaBatchRegresi = regresiObj.hitungBiaya(10, 50);

        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " + (int) biayaBatchRegresi);

    }

}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Modul06 ---
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
------------------------------------------------
BUILD SUCCESS
```

Penjelasan :

Program ini dirancang dengan menerapkan konsep inheritance (pewarisan) dan enkapsulasi, di mana kelas induk ModelKomputasi memuat atribut privat serta method dasar yang diwariskan ke kelas turunannya seperti ModelRegresi, ModelKlasifikasi, dan ModelDeepLearning. Setiap kelas turunan menggunakan keyword super untuk memanggil konstruktor kelas di atasnya dan melakukan overriding pada method tampilkanInfo() serta hitungBiaya() agar perhitungan spesifik setiap model dapat berjalan sesuai ketentuan.

Selain itu, program ini memanfaatkan konsep method overloading untuk variasi parameter method perhitungan biaya serta menerapkan polimorfisme di dalam kelas Main. Objek-objek dari berbagai kelas turunan disimpan ke dalam satu array bertipe

ModelKomputasi[] sehingga dapat dieksekusi secara efisien melalui perulangan untuk menampilkan informasi, mengakumulasi total biaya, serta menguji pemanggilan method khusus seperti epoch dan batch.

D. Kesimpulan

Praktikum Modul 6 mengenai Inheritance (Pewarisan) memberikan pemahaman mendalam mengenai proses penurunan sifat dari superclass ke subclass untuk menghindari duplikasi kode serta membentuk hubungan "is-a" antar objek. Melalui serangkaian kegiatan praktikum guided dan unguided, praktikan dapat mengaplikasikan penggunaan constructor kelas induk menggunakan kata kunci super, menerapkan method overriding dan method overloading, serta mengimplementasikan konsep polimorfisme melalui penggunaan array bertipe kelas induk.